

Docker



우리가 겪는 문제점

- ✔ "내 컴퓨터에만 안 깔려요": OS 차이, 보안 프로그램 충돌.
- ✔ 의존성 문제 : 파이썬 버전 불일치로 하루 종일 환경 설정만 하다가 끝나는 날들.
- ✔ 이식성 부족: 개발 환경에서 잘 되던 코드가 배포 서버에서는 작동하지 않음.

```
pip install -r requirements.txt
```

```
ERROR: Could not find a version that satisfies the  
requirement...
```



왜 Docker를 쓰는가?



이식성 (Portability)

프로그램과 환경을 하나의 패키지로 묶어, 어디서든 동일하게 실행할 수 있습니다.



일관성 (Consistency)

매번 귀찮은 설치 과정 없이, 항상 똑같은 환경을 즉시 구축할 수 있습니다.



독립성 (Isolation)

각 프로그램이 격리된 공간에서 실행되어 서로 충돌하지 않습니다.

VM vs Container

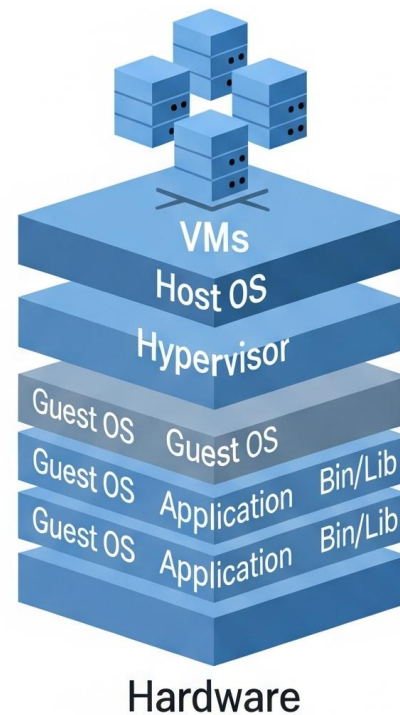
가상머신 (VM): 집을 새로 지음

각각의 앱마다 무거운 OS를 통째로 설치해야 합니다. 느리고 무겁습니다.

컨테이너 (Docker): 집 안에 구역을 나눔

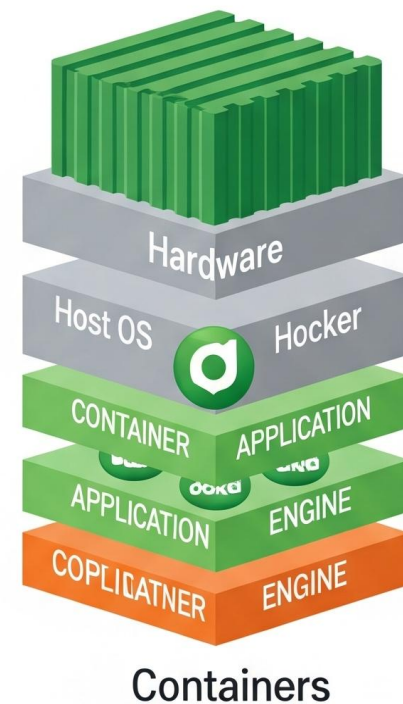
Host 컴퓨터의 핵심(Kernel)은 공유하고, 앱 실행에 필요한 최소한의 짐만 챙깁니다. 가볍고 빠릅니다.

Virtual Machines



- Resource Intensive
- Full OS Emulation
- Strong Isolation
- Slower Boot Times

Containers



- Lightweight
- Share Host OS Kernel
- Less Isolation
- Faster Boot Times

Enable
Application
Isolation

핵심 개념: 이미지 / 컨테이너



🖼️ 이미지 (Image)

닌텐도 게임 팩(설계도)과 같습니다. 실행에 필요한 모든 파일과 설정이 포함된 '읽기 전용' 파일입니다. (예: MySQL 설치파일+설정)

📦 컨테이너 (Container)

격리된 미니 컴퓨터 환경으로, 독립적인 환경으로 구성되고 각자의 IP와 포트를 가집니다.

설치 및 Nginx 실습

1. 설치 (Installation)

Docker Desktop을 다운로드하여 설치합니다. (Windows는 WSL, 가상화 설정 필요.)

<https://docs.docker.com/desktop/setup>

```
docker -v
```

```
Docker version 20.10.x, build...
```

2. 실행 (Run)

웹서버 Nginx를 다운받고 실행하여 4000번 포트로 연결합니다.

Nginx : 웹 서버, 역방향 프록시, 로드 밸런서, 캐시 서버, HTTP 캐싱 등 다양한 기능을 수행하는 오픈 소스 소프트웨어

```
docker run -d -p 4000:80 nginx
```

```
localhost:4000 접속 확인
```

```
docker rm -f [id]
```

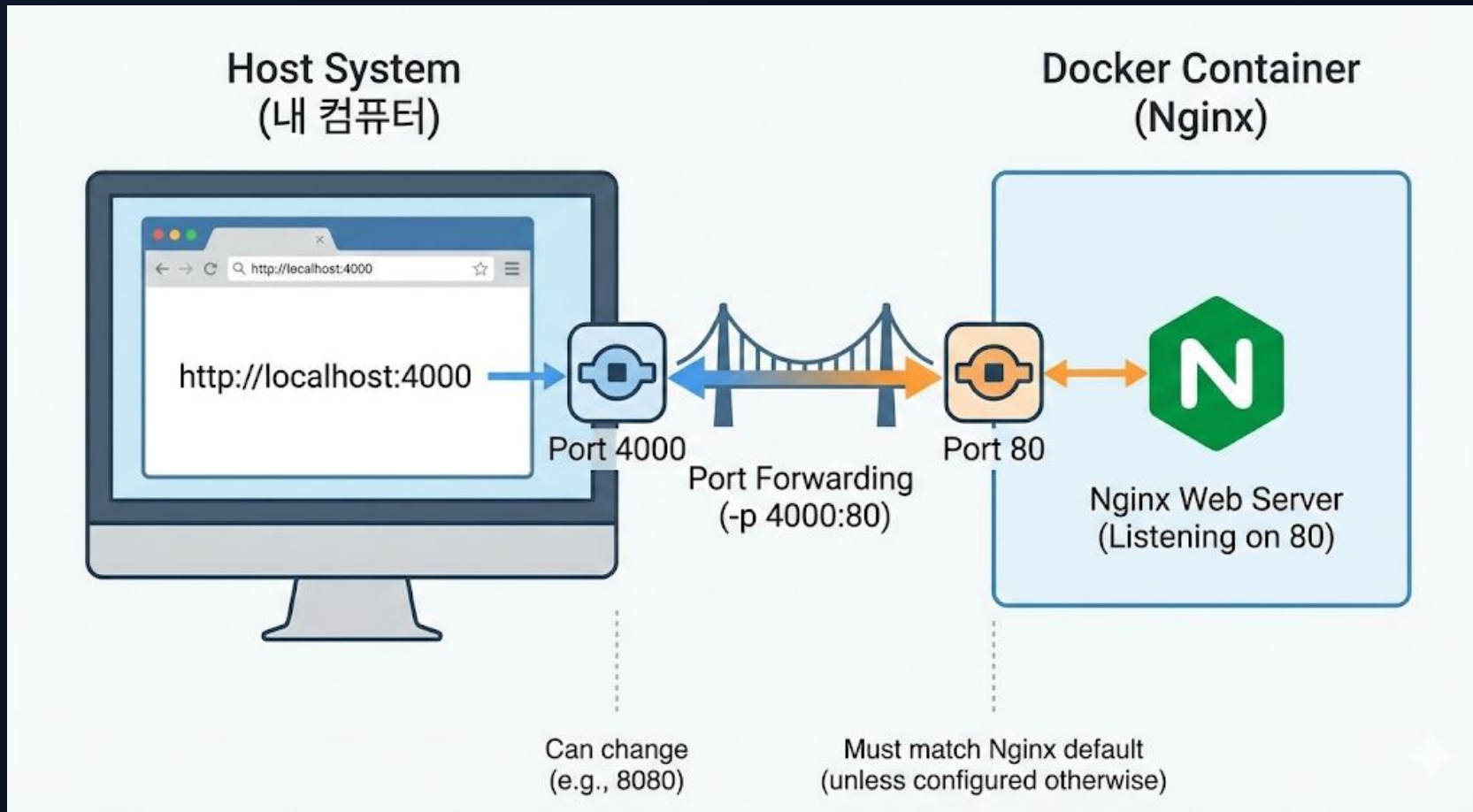
```
설치한 것 전부 삭제
```

```
docker ps -a
```

```
모든 컨테이너 조회
```

```
docker ps
```

```
실행중인 컨테이너 조회
```



컨테이너도 하나의 컴퓨터로 접속하려면 포트를
연결해야함
호스트 컴퓨터의 4000번 포트와 컨테이너의 80번 포트를
연결

Docker 라이프사이클

```
docker ps -a
```

모든 컨테이너 조회

```
docker ps
```

실행 중인 컨테이너 조회

1

Pull

이미지 다운로드
`docker pull nginx`

2

Create

컨테이너 생성
`docker create nginx`

3

Start

실행 (메모리 로드)
`docker start [컨테이너 id]`

4

Stop

중지
`docker stop [id]`

5

Rm

삭제
`docker rm [id]`

`docker run` 명령어는 Pull + Create + Start를 한 번에 수행합니다.

실행 모드 & 로그 조회

Foreground vs Background

✓ **Foreground (기본)**: 내 터미널을 차지합니다.

실시간으로 보이지만 다른 명령을 못 칩니다.

✓ **Background (-d 옵션)**: "Detached" 모드.

백그라운드에서 조용히 실행됩니다.

```
# 백그라운드 실행 (-d)
docker run -d nginx
```

숨은 로그 찾아보기 (Logs)

백그라운드로 실행하면 화면에 아무것도 안 보입니다.

이때 로그 명령어를 사용합니다.

```
# 1. 전체 로그 확인
docker logs [ID]

# 2. 마지막 10줄만 보기 (--tail)
docker logs --tail 10 [ID]

# 3. 실시간 로그 보기 (-f, follow)
docker logs -f [ID]
```

(나가기: **Ctrl + C**)

컨테이너 내부 접속

나만의 미니 컴퓨터 탐험

실행 중인 컨테이너는 독립된 리눅스 환경입니다. 직접 들어가서 내부 구조를 확인해봅시다.

exec -it: 터미널 모드로 접속

ls: 내부 파일 리스트 확인

cat: 설정 파일 내용 보기

```
# 1. 컨테이너 ID 확인 (docker ps)
docker exec -it [ID] bash

# 2. 내부에 접속된 상태
root@container:/# ls

bin dev etc home lib ...

root@container:/# cd /etc/nginx
root@container:/etc/nginx# cat nginx.conf

user nginx; worker_processes auto; ...
```

데이터 보존: Volume

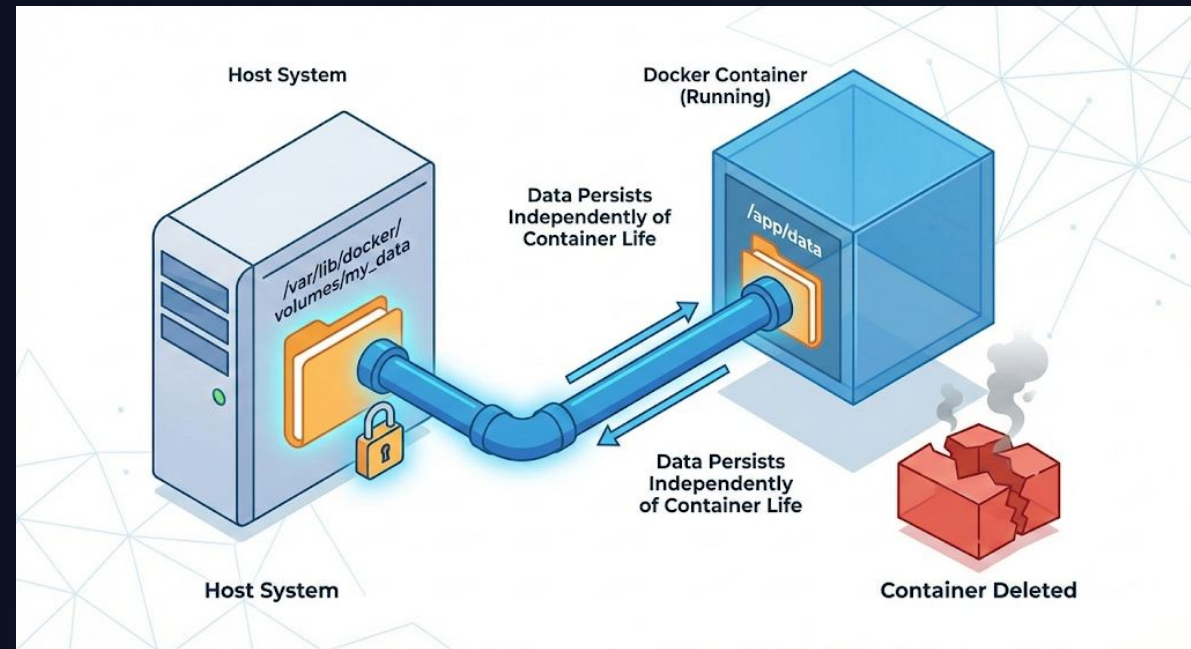
컨테이너는 휘발성입니다

이미지를 업데이트 하면 컨테이너가 재설치 됩니다. 그런데 컨테이너를 삭제하면 데이터도 함께 사라집니다. DB 같은 중요 데이터도 사라질 위험이 있습니다.

-v 옵션 사용

호스트(내 컴퓨터)의 폴더와 컨테이너 내부 폴더를 연결(마운트)하여 데이터를 호스트 컴퓨터에 영구적으로 저장합니다.

```
docker run -v [내경로]:[컨테이너경로] ...
```



파이썬 코드 이미지 만들기

1. 소스 코드 (main.py)

```
import time
while True:
    print("Hello Docker!", flush=True)
    time.sleep(3)
```

2. 설계도 (Dockerfile)

```
# 1. 베이스 이미지: 파이썬 3.9가 설치된 리눅스 환경을 가져온다.
FROM python:3.9

# 2. 작업 디렉토리: 컨테이너 내부의 /app 폴더를 작업 공간으로 쓴다.
WORKDIR /app

# 3. 파일 복사: 내 컴퓨터의 모든 파일(.)을 컨테이너 내부(.)로 복사한다.
COPY . .

# 4. 실행 명령어: 컨테이너가 시작될 때 이 명령어를 실행한다.
CMD ["python", "main.py"]
```

실행 모드 & 로그 조회

Foreground vs Background

✓ **Foreground (기본)**: 내 터미널을 차지합니다.

실시간으로 보이지만 다른 명령을 못 칩니다.

✓ **Background (-d 옵션)**: "Detached" 모드.

백그라운드에서 조용히 실행됩니다.

```
# 백그라운드 실행 (-d)
docker run -d nginx
```

숨은 로그 찾아보기 (Logs)

백그라운드로 실행하면 화면에 아무것도 안 보입니다.

이때 로그 명령어를 사용합니다.

```
# 1. 전체 로그 확인
docker logs [ID]

# 2. 마지막 10줄만 보기 (--tail)
docker logs --tail 10 [ID]

# 3. 실시간 로그 보기 (-f, follow)
docker logs -f [ID]
```

(나가기: **Ctrl + C**)

Docker Hub

github와 똑같이 이미지를 업로드하고 받을 수 있다

```
docker login  
docker tag my-app teacher/my-app:v1  
docker push teacher/my-app:v1
```

Docker Hub 실습 (배포하기)

1. 로그인

Docker Hub 웹사이트 가입 후 터미널에서 로그인합니다.

```
docker login
```

Username, Password 입력

웹에서 로그인 되어 있으면 자동 연결

2. 이름표 달기 (Tag)

택배 주소를 적는 것과 같습니다. '우리집 강아지' 대신 '서울시 강남구...'처럼 정확한 아이디/이미지명 형식이

필요합니다. `docker tag [기존이름] [내ID/새이름:버전]`

```
docker tag my-app teacher/my-app:v1
```

3. 업로드 (Push)

클라우드(Hub)로 이미지를 밀어 올립니다.

```
docker push teacher/my-app:v1
```

4. 공유 (Pull/Run)

코드나 파이썬 설치 없이 바로 실행 가능합니다.

```
docker run -d teacher/my-app:v1
```


Docker Compose 1. 개념 및 설치

컨테이너들의 지휘자 (composer)

지금까지는 컨테이너를 하나씩 따로 작동시켰습니다.

하지만 실제 서비스는 웹 서버, 데이터베이스, 캐시 서버 등 여러 컨테이너를 동시에 작동시켜야 합니다.

Docker Compose는 이 모든 컨테이너를 하나의 파일에 적고

명령어를 통해 한번에 작동시키게 해주는 도구입니다

설치 확인

Docker Desktop을 설치하셨다면 이미 설치되어 있습니다
(별도 설치 불필요)

```
docker-compose version
```

```
Docker Compose version v2.x.x...
```

```
('docker compose'로 하이픈 없이도 가능)
```

Docker Compose 2. 설계도 (YAML)

docker-compose.yml

프로젝트 루트 폴더에 이 파일을 만들면 준비 끝입니다.
들어쓰기가 매우 중요합니다

- ✓ **version:** 컴포즈 파일의 버전
- ✓ **services:** 실행할 컨테이너 목록 (web, db 등)
- ✓ **image:** 사용할 이미지 이름
- ✓ **ports:** 포트 연결 ("호스트:컨테이너")
- ✓ **environment:** 환경변수 설정

```
version: '3.8'

services:

  my-web-server:

    # 컨테이너 1

    image: nginx

    ports:

      - "8080:80"

  my-database:

    # 컨테이너 2

    image: mysql:5.7

    environment:

      MYSQL_ROOT_PASSWORD: 1234
```

Docker Compose 3. 실행과 종료 (Lifecycle)

실행 및 종료

명령어 한 줄로 수십 개의 컨테이너를 동시에 제어합니다.

- ✓ **up**: 생성 및 실행 (Create & Start)
- ✓ **down**: 멈추고 삭제 (Stop & Remove)
- ✓ **stop/start**: 잠깐 멈춤 / 재개

1. 실행 (가장 많이 씀)

```
docker-compose up -d
```

-d: 백그라운드 실행, 이미지가 없으면 다운로드까지 자동

2. 종료 및 정리 (끝낼 때)

```
docker-compose down
```

컨테이너와 네트워크를 모두 삭제 (데이터는 볼륨에 보존)

3. 잠시 멈춤 (삭제 X)

```
docker-compose stop
```

Docker Hub 실습 (명령어들)

1. 상태 (PS)

이 프로젝트의 컨테이너들만 보여줍니다.

```
docker-compose ps
```

현재 실행 중인 서비스 목록

2. 로그 (Logs)

특정 서비스의 로그만 선택하여 봅니다.

```
docker-compose logs -f my-database
```

데이터베이스 서비스의 로그만 실시간 확인

3. 재빌드 (Build)

코드가 수정되었다면 이미지를 다시 생성해야 합니다.

```
docker-compose up -d --build
```

이미지를 새로 빌드하고 실행

4. 재시작 (Restart)

설정을 바꾸거나 에러가 났을 때.

```
docker-compose restart
```

전체 또는 특정 서비스 재시작

환경변수 설정

도커에서 환경변수(Environment Variables)는 컨테이너 안의 프로그램 동작을 바꾸거나 비밀번호 /API 키 같은 민감한 정보를 전달할 때 사용합니다.

1. 간단하게 한두 개 설정할 때 (-e)

```
docker run -e [변수명]=[값] [이미지]
```

```
# 비밀번호를 '1234'로 설정하여 실행
docker run -d --name mydb -e MYSQL_ROOT_PASSWORD=1234 mysql
```

2. 변수가 많아서 파일로 관리할 때 (--env-file)

```
# 1. 파일 생성(.env를 루트파일에 생성)
MYSQL_ROOT_PASSWORD=1234
MYSQL_DATABASE=myapp
MYSQL_USER=user
```

```
# 2. 실행 명령어
docker run -d --env-file .env mysql
```

3. Docker Compose(가장 추천)

방식 A: 직접 작성하기

```
version: '3.8'
services:
  db:
    image: mysql
    environment:
      - MYSQL_ROOT_PASSWORD=1234
      - TIMEZONE=Asia/Seoul
```

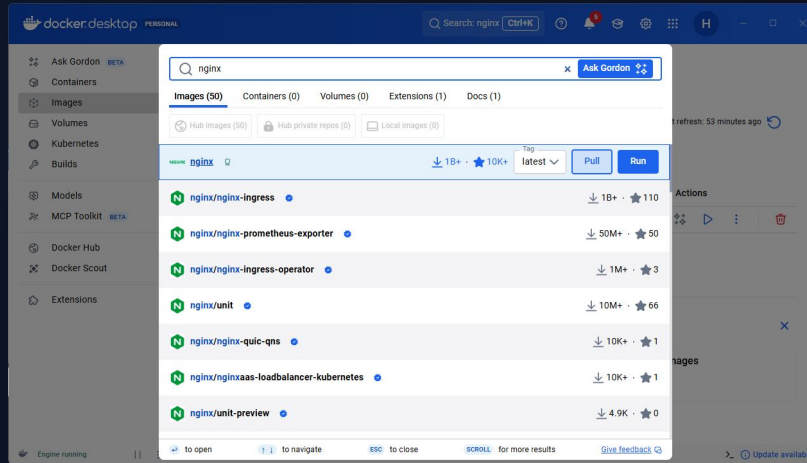
방식 B: 파일 불러오기

```
version: '3.8'
services:
  db:
    image: mysql
    env_file:
      - .env # 같은 폴더에 있는 .env 파일을 읽어옴
```

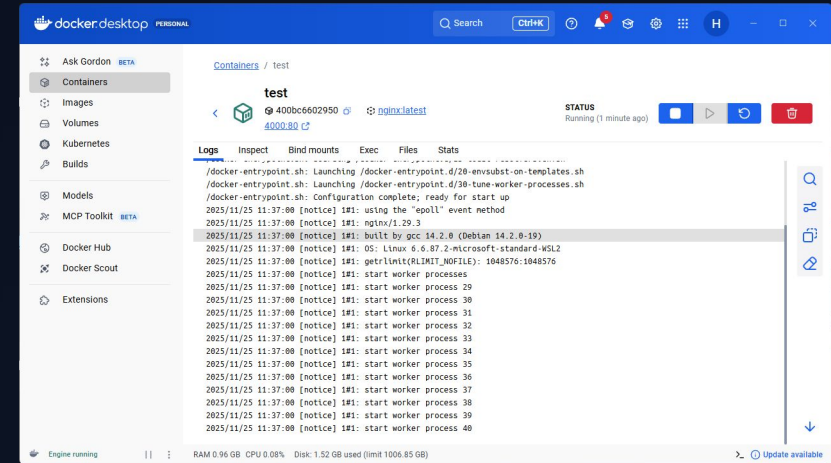
Docker Desktop

Docker Desktop은 직관적인 그래픽 화면(GUI)을 제공합니다. 이미지 검색, 컨테이너 관리, 볼륨 생성 등 모든 기능을 사용 가능합니다

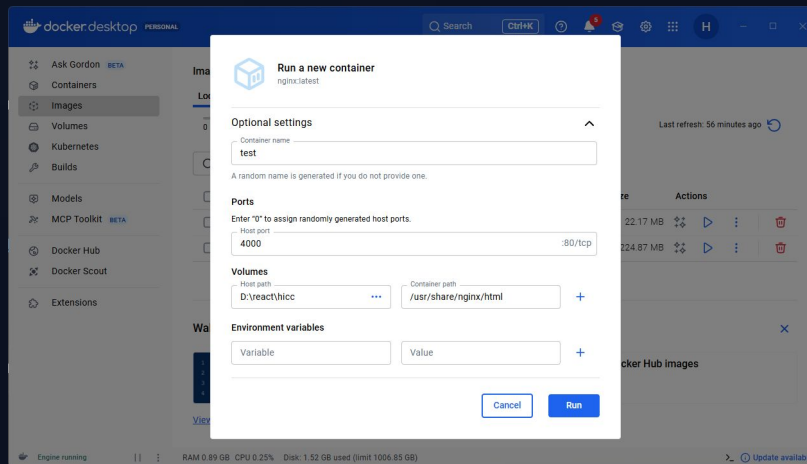
1. 이미지 검색 및 설치



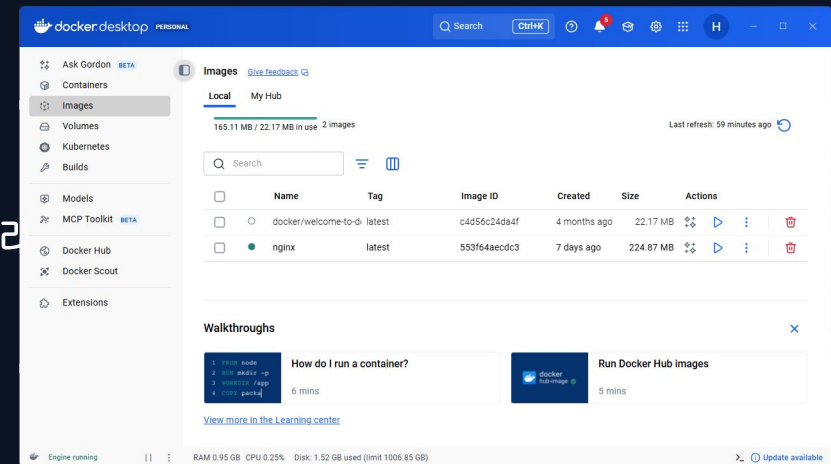
3. 컨테이너 관리



2. 컨테이너 생성



4. 설치한 이미지 관리



실습 뒷정리 (Clean Up)

하나씩 지우기 (정석)

실습했던 컨테이너와 이미지를 순서대로 삭제합니다. 실행 중인 컨테이너는 먼저 멈춰야 합니다.

```
# 1. 컨테이너 중지  
docker stop py-con
```

```
# 2. 컨테이너 삭제  
docker rm py-con
```

```
# 3. 이미지 삭제 (필요 없다면)  
docker rmi my-app
```

Docker Desktop에서도 가능합니다

한방에 지우기 (개발용)

귀찮을 때 사용하는 강제 명령어입니다.

모든 컨테이너 중지 (\$()는 명령어의 결과를 넣는 문법)

```
docker stop $(docker ps -qa)
```

모든 컨테이너 삭제

```
docker rm $(docker ps -qa)
```

사용하지 않는 모든 이미지, 캐시 정리

```
docker system prune -a
```